Universidad Mariano Gálvez de Guatemala

Facultad de Ingeniería

Centro Universitario de Escuintla



CARRERA

Ingeniería en Sistemas

CURSO

Sistemas Operativos I

CATEDRATICO

Ing. Carlos Gómez

INTEGRANTES DEL GRUPO

Domenick Emmanuel Montepeque Cifuentes 2190-24-5736
Arnoldo Esteban Morales Salvador 2190-24-11268
Cristian Ronaldo Navarro García 2190-24-23371
Santiago Elsias Jax Soc 2130-24-23883
Rony Milton Yanes Ruiz 2190-24-1980

PROYECTO FINAL:

Simulador visual de administración de recursos de un sistema operativo

Escuintla, Guatemala, 18 de octubre de 2026

Introducción

Los sistemas operativos administran los recursos de la computadora (procesador, memoria y dispositivos) y deciden cómo se reparten entre los procesos que compiten por ellos. Comprender esas decisiones es difícil si solo se estudian de forma teórica, porque los algoritmos de planificación, la sincronización de hilos, los interbloqueos y la asignación de memoria dependen de datos y de tiempos que cambian en cada ejecución (Silberschatz et al., 2018; Tanenbaum y Bos, 2015).

Este informe documenta el desarrollo de un simulador visual e interactivo, escrito en Java con la biblioteca gráfica Swing, que integra cinco componentes: gestión de procesos, planificación del procesador, concurrencia y sincronización, interbloqueos y gestión de memoria. El objetivo no es construir un sistema operativo completo, sino contar con una herramienta que permita observar y analizar el comportamiento de los mecanismos estudiados en el curso, tal como lo establece el documento del proyecto final (Universidad Mariano Gálvez de Guatemala, 2026).

El documento presenta los objetivos, el alcance, la arquitectura de la solución, la descripción técnica de cada módulo, las pruebas realizadas con sus resultados, y las conclusiones y recomendaciones del equipo.

Objetivos

Objetivo general

Desarrollar una aplicación visual que permita simular y analizar mecanismos de administración de recursos de un sistema operativo, mediante la implementación de algoritmos de planificación de procesos, mecanismos de concurrencia y sincronización, manejo de interbloqueos y estrategias de gestión de memoria.

Objetivos específicos

1. Representar procesos y sus principales atributos mediante una interfaz gráfica.
2. Implementar y comparar los algoritmos de planificación FCFS, SJF y Round Robin.
3. Demostrar una condición de carrera y aplicar mecanismos de exclusión mutua y sincronización.
4. Simular una situación de interbloqueo e implementar una estrategia de evitación mediante el algoritmo del banquero.
5. Representar la asignación y liberación de memoria con las estrategias First Fit, Best Fit y Worst Fit, e identificar la fragmentación resultante.
6. Documentar el desarrollo técnico y el proceso de trabajo grupal mediante los instrumentos institucionales requeridos.

Descripción y alcance del proyecto

El simulador permite al usuario registrar procesos con su identificador, tiempo de llegada, ráfaga de CPU, prioridad y memoria solicitada, y ejecutar sobre ellos los distintos módulos. La interfaz está organizada en cinco módulos a los que se accede desde un menú lateral: planificador de CPU, memoria RAM, interbloqueos, concurrencia, y bloque de control de proceso (PCB) con su ciclo de vida.

El alcance del proyecto incluye:

- Los algoritmos de planificación FCFS, SJF no expropiativo y Round Robin con quantum configurable por el usuario.
- El cálculo de los tiempos de espera, retorno y respuesta, con sus promedios, y el diagrama de Gantt.

- Una demostración de acceso concurrente a un recurso compartido con y sin sincronización.
- El algoritmo del banquero para determinar si un estado es seguro.
- La asignación de memoria por particiones fijas con tres estrategias de ajuste.

Quedan fuera del alcance la paginación, la memoria virtual y los algoritmos de reemplazo de páginas, que el documento del proyecto considera mejoras opcionales.

Arquitectura de la solución y tecnologías utilizadas

Tecnologías

La aplicación se desarrolló en Java, compilada con el kit de desarrollo JDK 21 (Eclipse Temurin), y su interfaz gráfica utiliza Swing. Se empleó Visual Studio Code como editor y Git como sistema de control de versiones. Para la sincronización se utilizaron las clases `ReentrantLock` y `Semaphore` del paquete `java.util.concurrent` (Oracle, s. f.).

Estructura del código

La solución separa la lógica de cada módulo de la interfaz gráfica, de modo que los algoritmos puedan probarse de forma independiente. La Tabla 1 resume las clases del proyecto.

Tabla 1

Clases del simulador y su responsabilidad

Clase	Responsabilidad
Main	Punto de entrada. Lanza la interfaz en el hilo de eventos de Swing.
VentanaPrincipal	Interfaz gráfica con los cinco módulos, su menú lateral y la lectura de datos de entrada.
Proceso	Modelo del proceso: atributos, estado y tiempos calculados.
PlanificadorCPU	Algoritmos FCFS, SJF y Round Robin; genera el diagrama de Gantt y el reporte de tiempos.
BloqueMemoria	Modelo de una partición de memoria y su fragmentación interna.
GestorMemoria	Estrategias First Fit, Best Fit y Worst Fit; liberación de memoria.
GestorDeadlock	Matrices de asignación, máximo y necesidad; algoritmo del banquero.
MonitorConcurrencia	Escenarios con hilos: sin sincronización, con mutex y con semáforos.
InspectorPCB	Visualización del PCB y simulación del cambio de contexto.

Descripción técnica de cada módulo

Gestión de procesos

La clase Proceso representa el bloque de control de proceso. Almacena el identificador (PID), el tiempo de llegada, la ráfaga, la prioridad, la memoria solicitada y el estado, que puede tomar los valores Nuevo, Listo, Ejecución, Bloqueado y Terminado. También guarda los tiempos calculados por los algoritmos: inicio, finalización, retorno, espera y respuesta. El tiempo de retorno se calcula como la diferencia entre el tiempo de finalización y el de llegada, y el tiempo de espera como el retorno menos la ráfaga (Silberschatz et al., 2018).

La clase InspectorPCB muestra los campos del PCB (PID, estado, contador de programa, prioridad, memoria asignada y ráfaga restante) y simula un cambio de contexto: guarda el estado del proceso que sale y carga el del que entra.

Planificación del procesador

`PlanificadorCPU` implementa tres algoritmos:

- **FCFS.** Ordena los procesos por tiempo de llegada y los ejecuta en ese orden, sin interrupción.
- **SJF no expropiativo.** En cada instante en que el procesador queda libre, elige entre los procesos ya llegados el de menor ráfaga.
- **Round Robin.** Mantiene una cola de procesos listos y asigna a cada uno un máximo de tiempo igual al quantum. Si el proceso no termina, regresa al final de la cola. El quantum lo define el usuario.

Si el procesador queda sin procesos listos, el algoritmo registra un intervalo ocioso (IDLE) en el diagrama de Gantt. Para cada ejecución se calculan los promedios de retorno, espera y respuesta.

En la interfaz, el usuario elige el algoritmo y, en Round Robin, define el quantum. El diagrama de Gantt se dibuja con una barra de color por proceso y los tiempos de inicio y fin debajo, y los promedios de retorno, espera y respuesta se muestran en tres indicadores.

Concurrencia y sincronización

`MonitorConcurrencia` crea varios hilos que incrementan un contador compartido. Se implementaron cuatro escenarios:

1. **Sin sincronización.** Cada hilo lee el contador, espera un milisegundo y escribe el valor leído más uno. Esta pausa hace evidente la condición de carrera, porque varios hilos escriben a partir de una lectura que ya quedó desactualizada.
2. **Con mutex.** La sección crítica se protege con `ReentrantLock`.

3. **Con semáforo binario.** Se usa un Semaphore con un solo permiso mediante las operaciones acquire (wait) y release (signal).
4. **Con semáforo contador.** Se simula un conjunto de tres recursos y se muestra cómo cambia el valor del semáforo cuando varios hilos lo solicitan.

En la interfaz, el usuario define el número de hilos y de incrementos por hilo, y el módulo ejecuta los escenarios sin sincronización y con mutex. Los resultados se muestran lado a lado: el valor esperado, el valor obtenido y un aviso que indica si hubo condición de carrera. Los botones de semáforo binario y semáforo contador muestran su salida en la consola del módulo.

Interbloqueos

Un interbloqueo ocurre cuando un conjunto de procesos queda esperando indefinidamente recursos que retienen entre sí. Requiere cuatro condiciones simultáneas: exclusión mutua, retención y espera, no apropiación y espera circular (Silberschatz et al., 2018). En el caso simulado, dos procesos retienen un recurso cada uno y solicitan el que tiene el otro: los recursos no se pueden compartir (exclusión mutua), cada proceso conserva lo que tiene mientras espera (retención y espera), nadie puede quitarle el recurso a otro (no apropiación), y la cadena de espera se cierra sobre sí misma (espera circular).

La estrategia aplicada es la **evitación**, mediante el algoritmo del banquero. GestorDeadlock recibe el vector de recursos disponibles y las matrices de asignación y máximo, calcula la matriz de necesidad (necesidad = máximo − asignación) y busca una secuencia segura: un orden en el que cada proceso pueda terminar con los recursos disponibles, que se incrementan al liberar los que ya terminaron. Si en alguna ronda ningún proceso puede continuar, el estado se declara inseguro y se listan los procesos bloqueados.

En la interfaz, el usuario elige uno de tres escenarios (un caso seguro con cinco procesos y tres recursos, un interbloqueo de dos procesos y uno circular de tres) y puede editar las matrices de asignación, máximo y disponibles antes de evaluar. El programa dibuja el grafo de asignación de recursos: los procesos son círculos, los recursos son cuadros, las flechas verdes indican los recursos asignados y las flechas rojas punteadas las solicitudes de los procesos bloqueados. También muestra un aviso con el veredicto, analiza las cuatro condiciones del interbloqueo y sugiere una recuperación posible.

Gestión de memoria

La memoria se modela como un conjunto de particiones fijas (`BloqueMemoria`), cada una con su tamaño, su estado (libre u ocupada) y el proceso asignado. `GestorMemoria` implementa tres estrategias (Tanenbaum y Bos, 2015):

- **First Fit.** Asigna la primera partición libre en la que el proceso cabe.
- **Best Fit.** Asigna la partición libre más pequeña en la que el proceso cabe, es decir, la que deja el menor sobrante.
- **Worst Fit.** Asigna la partición libre más grande, la que deja el mayor sobrante.

Al asignar, se calcula la fragmentación interna como el tamaño de la partición menos el tamaño del proceso. Al terminar un proceso, su partición se libera.

En la interfaz, el usuario escribe los tamaños de las particiones, elige la estrategia y asigna los procesos registrados en el módulo de planificación. Puede liberar un proceso por su PID o reiniciar la memoria. El mapa dibuja cada partición con su proceso, la fragmentación interna en amarillo y las particiones libres en gris, y muestra los totales de memoria usada, fragmentación interna y particiones libres.

Pruebas realizadas y resultados

Las pruebas se hicieron ejecutando las clases de lógica y la interfaz gráfica con datos controlados. Los valores de las tablas son la salida real del programa.

Planificación del procesador

Se usaron cuatro procesos con los datos de la Tabla 2.

Tabla 2

Procesos de prueba para la planificación

PID	Llegada (ms)	Ráfaga (ms)
P1	0	5
P2	1	3
P3	2	8
P4	3	6

La Tabla 3 compara los promedios obtenidos con cada algoritmo.

Tabla 3

Tiempos promedio por algoritmo de planificación (ms)

Algoritmo	Retorno	Espera	Respuesta
FCFS	11.25	5.75	5.75
SJF no expropiativo	10.75	5.25	5.25
Round Robin (q = 2)	15.25	9.75	2.00
Round Robin (q = 4)	14.75	9.25	4.00

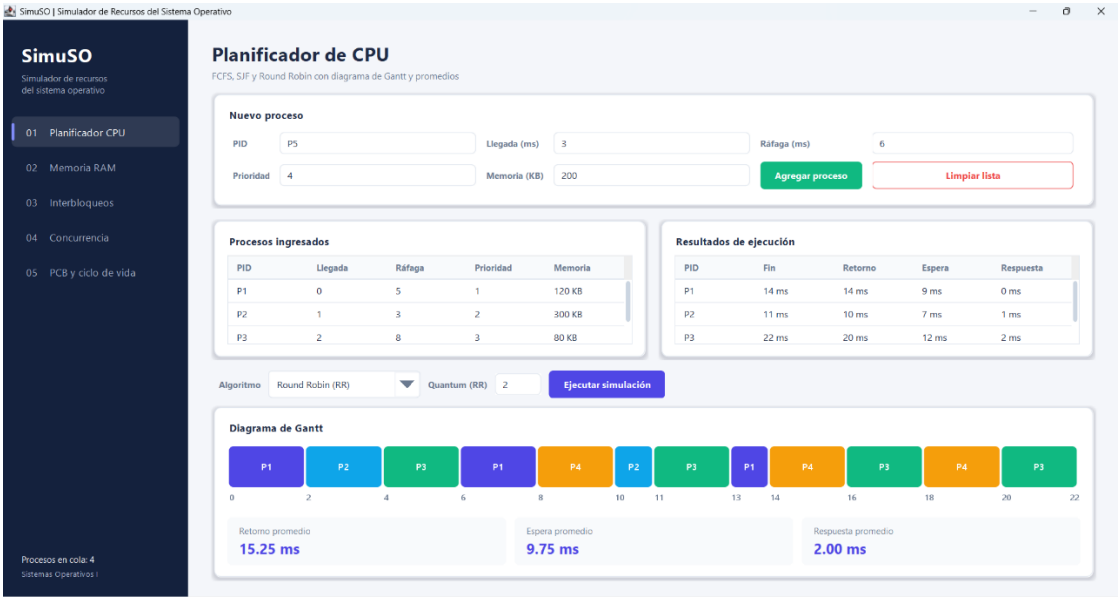
El orden de ejecución fue P1, P2, P3, P4 en FCFS (0–5, 5–8, 8–16 y 16–22 ms) y P1, P2, P4, P3 en SJF (0–5, 5–8, 8–14 y 14–22 ms). Con SJF, el proceso P4 (ráfaga 6) se ejecutó antes que P3 (ráfaga 8) y por eso bajó el tiempo promedio de espera de 5.75 a 5.25 ms.

Round Robin obtuvo el mejor tiempo de respuesta (2.00 ms con q = 2), porque cada proceso recibe el procesador poco después de llegar, pero tuvo los peores promedios de espera y retorno por los cambios entre procesos. Al aumentar el quantum de 2 a 4, el tiempo de respuesta

empeoró (de 2.00 a 4.00 ms) y los de espera y retorno mejoraron levemente, lo que muestra la relación entre el quantum y el rendimiento del algoritmo.

Figura 1

Diagrama de Gantt con Round Robin (q = 2)



Nota. Captura del simulador con los procesos de la Tabla 2.

Gestión de memoria

Se configuraron cinco particiones de 100, 500, 200, 300 y 600 KB, y se asignaron cuatro procesos de 120, 300, 80 y 200 KB, en ese orden. Los resultados se muestran en la Tabla 4.

Tabla 4

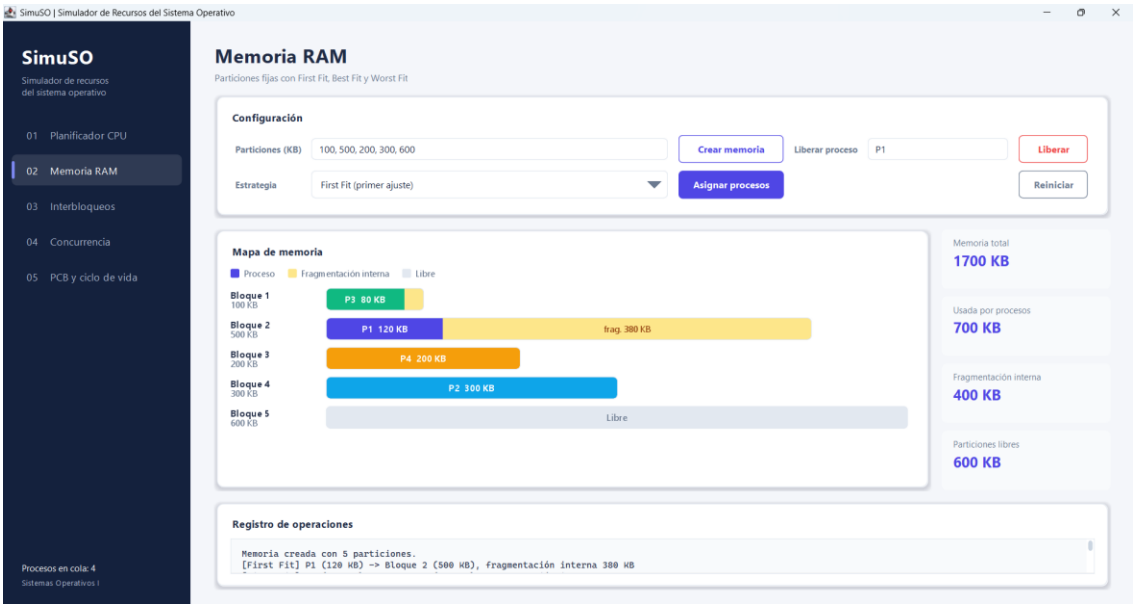
Asignación de memoria y fragmentación interna por estrategia (KB)

Estrategia	P1 (120)	P2 (300)	P3 (80)	P4 (200)	Fragmentación interna total	Partición libre al final
First Fit	Bloque 2 (500)	Bloque 4 (300)	Bloque 1 (100)	Bloque 3 (200)	400	Bloque 5 (600 KB)
Best Fit	Bloque 3 (200)	Bloque 4 (300)	Bloque 1 (100)	Bloque 2 (500)	400	Bloque 5 (600 KB)
Worst Fit	Bloque 5 (600)	Bloque 2 (500)	Bloque 4 (300)	Bloque 3 (200)	900	Bloque 1 (100 KB)

First Fit y Best Fit produjeron la misma fragmentación total (400 KB), pero repartida de forma distinta. Worst Fit generó 900 KB, más del doble, y dejó libre solo la partición de 100 KB, que no admitiría un proceso grande, mientras que con las otras dos estrategias queda libre la partición de 600 KB. En este caso, Worst Fit fue la estrategia menos eficiente.

Figura 2

Mapa de memoria con la estrategia First Fit



Nota. Captura del simulador con particiones de 100, 500, 200, 300 y 600 KB, y los procesos de la Tabla 2 (P1 = 120 KB, P2 = 300 KB, P3 = 80 KB y P4 = 200 KB). La fragmentación interna total es de 400 KB y queda libre la partición de 600 KB.

Interbloqueos

Se probaron dos casos con el algoritmo del banquero.

Caso seguro. Se usaron cinco procesos y tres tipos de recursos, con disponibles [3, 3, 2], asignación [[0,1,0], [2,0,0], [3,0,2], [2,1,1], [0,0,2]] y máximo [[7,5,3], [3,2,2], [9,0,2], [2,2,2], [4,3,3]]. La matriz de necesidad resultó [[7,4,3], [1,2,2], [6,0,0], [0,1,1], [4,3,1]]. El algoritmo

encontró la secuencia segura <P1, P3, P4, P0, P2> y el vector de disponibles terminó en [10, 5, 7].

Caso inseguro. Se usaron dos procesos y dos recursos, con disponibles [0, 0], asignación [[1,0], [0,1]] y máximo [[1,1], [1,1]]. Cada proceso necesita el recurso que tiene el otro. Ninguno pudo avanzar, y el sistema se declaró inseguro con P0 y P1 bloqueados, lo que corresponde a una espera circular.

Concurrencia y sincronización

La Tabla 5 presenta los resultados de los escenarios con cinco hilos.

Tabla 5

Resultados de los escenarios de concurrencia

Escenario	Incrementos por hilo	Valor esperado	Valor obtenido
Sin sincronización	20	100	20
Con mutex	1000	5000	5000
Con semáforo binario	1000	5000	5000

Sin sincronización se perdieron 80 de los 100 incrementos, lo que evidencia la condición de carrera. Este valor puede variar entre ejecuciones porque depende de la planificación de los hilos. Con mutex y con semáforo binario, el resultado coincidió con el esperado en todas las ejecuciones realizadas.

En la demostración del semáforo contador con tres permisos, los hilos 1, 2 y 3 obtuvieron acceso y el valor del semáforo bajó de 3 a 0. Los hilos 4 y 5 quedaron bloqueados. Al liberar dos permisos, el valor del semáforo pasó a 2.

Conclusiones y recomendaciones

Conclusiones

1. Ningún algoritmo de planificación es mejor en todos los criterios. Con los datos de prueba, SJF obtuvo los menores tiempos de espera y retorno, y Round Robin el menor tiempo de respuesta.
2. El quantum de Round Robin influye directamente en el resultado: un valor menor mejoró el tiempo de respuesta, pero aumentó los cambios entre procesos.
3. La falta de sincronización produjo resultados incorrectos de forma evidente, mientras que el mutex y el semáforo binario garantizaron la exclusión mutua.
4. El algoritmo del banquero distinguió correctamente un estado seguro de uno inseguro, y permitió relacionar el caso con las cuatro condiciones necesarias del interbloqueo.
5. En la prueba de memoria, la estrategia de asignación cambió de forma importante la fragmentación (400 KB frente a 900 KB), por lo que la elección del método tiene un efecto medible.
6. Separar la lógica de los algoritmos de la interfaz gráfica facilitó probar cada módulo de manera independiente.

Recomendaciones

1. Agregar paginación y algoritmos de reemplazo de páginas para ampliar el módulo de memoria.
2. Incorporar planificación expropiativa por prioridad y SRTF para comparar más algoritmos.
3. Permitir agregar o quitar procesos y recursos en el módulo de interbloqueos, para probar sistemas de cualquier tamaño.

4. Repetir las pruebas de concurrencia varias veces y reportar el promedio, dado que sus resultados varían entre ejecuciones.

Referencias

Oracle. (s. f.). *Java Platform, Standard Edition 21 API specification*. https://docs.oracle.com/en/java/javase/21/docs/api/

Silberschatz, A., Galvin, P. B., y Gagne, G. (2018). *Operating system concepts* (10.ª ed.). Wiley.

Tanenbaum, A. S., y Bos, H. (2015). *Modern operating systems* (4.ª ed.). Pearson.

Universidad Mariano Gálvez de Guatemala. (2026). *Proyecto final: Simulador visual de administración de recursos de un sistema operativo* [Documento del curso]. Facultad de Ingeniería en Sistemas.

Anexos

Anexo A. Instrucciones de ejecución

1. Instalar el JDK 21 o superior.
2. Abrir una terminal en la carpeta del proyecto.
3. Compilar con `javac -encoding UTF-8 *.java`.
4. Ejecutar con `java Main`.

Anexo B. Enlace al repositorio

https://github.com/amoraless36-stack/simulador-so-umg